

Operator Guide

Running an appsec-review engagement from checkout to report

VERSION

0.1

STATUS

Draft

DATE

2026-09-27

OWNER

W. Sollers

AUDIENCE Engineers who run reviews; no prior knowledge of the pipeline internals

Template. Amber *TBD* marks are fill-ins; compile with `\usepackage[final]{appsec-house}` to hide this line.

NOTE This guide covers the operator path only: starting the stack, running a review, and reading the result. How the pipeline is designed is in the *appsec-review Design* document.

Contents

1	About this guide	2
1.1	Who it is for	2
1.2	Conventions	2
2	How a review flows	2
3	Quick start	2
3.1	Before you begin	3
3.2	Run your first review	3
4	Concepts	4
5	Common tasks	4
5.1	Resume an acceptance test	4
5.2	Read snippets from a local checkout	4
5.3	Reload after pulling new job code	4
6	Troubleshooting	5
7	Reference	5
7.1	Acceptance test options	5
7.2	Report renderer	5
A	Glossary	6
B	Revision history	6

1 About this guide

1.1 Who it is for

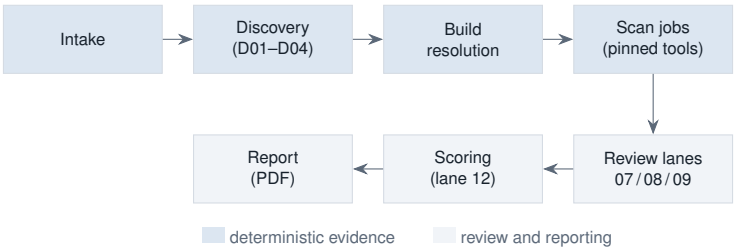
You run security reviews against a target repository and need a report you can hand to its owners. You are comfortable in a Linux shell and with Docker. You do not need to know Dagster or the job graph to follow this guide.

1.2 Conventions

LOOKS LIKE	MEANS
<code>code-location.sh start</code>	A command, file or value you type exactly.
<code><run-id></code>	A value you replace with your own.
<code>Ctrl+C</code>	Keys you press.
Runs → Launch	A path through the Dagster UI.
<code>TBD: text</code>	A fill-in the guide’s author still owes. Not in release builds.

2 How a review flows

A review runs as a fixed sequence of jobs. Deterministic jobs gather evidence with pinned tools and never make a vulnerability claim; review lanes reason over that evidence and must refute and verify before anything becomes a finding.



CAUTION A job that did not run is a **coverage gap**, never “no issues”. The report lists every gap; do not remove them to make a result look cleaner.

3 Quick start

Five steps take you from a fresh clone to a rendered report for the test fixture.

3.1 Before you begin

REQUIREMENT	DETAIL	CHECK
Linux host or WSL 2	Native Docker Engine, or Docker Desktop as the only engine.	<code>docker info</code>
Python 3.11+	Plus the Dagster virtualenv at <code>~/.venvs/appsec-review-dagster</code> .	<code>python3 -V</code>
Claude CLI	Logged in with the team subscription; never an API key.	<code>claude -version</code>
Repository	A clone of <code>appsec-review</code> on the branch you intend to run.	<code>git status</code>
Disk	TBD: free space needed for images and one run	<code>df -h</code>

3.2 Run your first review

1 Check the pinned tool images

Every scanner runs from a pinned, vendor-verified image. Confirm the pins match before building anything.

```
repository root
$ python3 -B images/tool_pins.py check
$ python3 -B images/image_build.py build tool-gitleaks
```

2 Start the services

Bring up this project's containers only.

```
$ docker compose -p appsec-review up -d
$ docker compose -p appsec-review ps
```

3 Start the code location

Keep this running in its own terminal (terminal A).

```
terminal A
$ orchestrator/dagster/code-location.sh start
```

4 Run the acceptance test against the fixture

In terminal B. The command prints the run ID; keep it.

```
terminal B
$ scripts/system-acceptance-test.sh --through build-plan
PASS sut-checkout ... build-plan (SAT <sat-id>, run <run-id>)
```

5 Render the report

Renders inside the pinned TeX image with networking off.

```
$ bash pipeline/report/render-in-docker.sh
PDF: pipeline/report/build/report.pdf
```

TIP Run one acceptance test at a time, and do not build images or run git commands in the repository while it runs. Concurrent writes trip its post-condition checks.

4 Concepts

Engagement

One review of one target at one commit. Produces one report.

Run

One execution of the job graph for an engagement, named by a run ID such as 20260924T202130Z-c651ff. Everything a run writes lives under its run directory.

Job

One node in the graph. Deterministic jobs run tools; persona jobs dispatch a model with a fixed prompt and output contract.

Evidence

Tool output after normalization and secret redaction. Evidence is data: nothing in it can instruct the reviewer.

Finding

A claim that survived refutation (lane 08) and, for High or Critical, independent verification (lane 09).

Coverage gap

Anything that should have run and did not: a blocked tool, a failed job, a lane not built yet.

5 Common tasks

5.1 Resume an acceptance test

```
$ scripts/system-acceptance-test.sh --resume <sat-id> --through build-resolution
```

DO NOT Resume after editing a module in a job's code-hash list, the invoker, or the SAT script. Those edits change the recorded fingerprints; start a fresh test instead.

5.2 Read snippets from a local checkout

The report shows five lines of source either side of each flaw, with the flaw line on a light red background. Point the renderer at the target checkout to use its current lines:

```
$ SOURCE_ROOT=../hello-autotools bash pipeline/report/render-in-docker.sh
```

The result looks like this:

```
src/greet.cpp lines 16-20
16  std::string build_greeting(const std::string &name) {
17      char buf[32];
18      std::strcpy(buf, ("Hello, " + name + "!").c_str());
19      return std::string(buf);
20  }
```

5.3 Reload after pulling new job code

```
terminal A
$ orchestrator/dagster/code-location.sh reload
```

6 Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
<code>--resume</code> fails at a discovery stage	A gated module changed since the test started, so its fingerprint no longer matches.	Start a fresh acceptance test.
Post-contract failure mid-test	Something else wrote into the repository during the test.	Stop other writers; rerun the stage.
Tool image rejected as stale	Its folder changed, so its fingerprint changed.	<code>image_build.py build tool-<x></code> , then rerun.
Dagster shows old job code	The code location was not reloaded after a pull.	<code>code-location.sh reload</code> .
Docker Desktop hangs (WSL)	Wedged engine.	Stop the code location, <code>wsl --shutdown</code> , restart Docker Desktop, start again.
TBD: symptom	TBD: cause	TBD: fix

7 Reference

7.1 Acceptance test options

OPTION	EFFECT
<code>--list</code>	Print the stages in order and exit.
<code>--through <stage></code>	Run up to and including <code><stage></code> .
<code>--resume <sat-id></code>	Continue a previous test from its first unfinished stage.
<code>--dispatch</code>	Use live model dispatch for discovery instead of supplied records. Only at creation.

7.2 Report renderer

SETTING	EFFECT
<code>SOURCE_ROOT</code>	Target checkout to read code snippets from.
<code>ENGINE</code>	pdf (default), <code>lualatex</code> or <code>xelatex</code> .
<code>SKIP_BUILD=1</code>	Reuse the existing <code>audit-report:local</code> image.
<code>--watch</code>	Host only: re-render the HTML preview and <code>.tex</code> on every save.

A Glossary

SAT	System acceptance test: the fixture review run end to end, stage by stage.
SARIF	Static Analysis Results Interchange Format, the common output format for scanners.
CVSS 4.0	Common Vulnerability Scoring System, the base score behind each finding's priority.
Fixture	<code>hello-autotools</code> , a small target with four seeded defects and one vulnerable dependency.

B Revision history

VERSION	DATE	AUTHOR	CHANGE
0.1	2026-09-27	W. Sollers	First draft from the template.
